

SYSTEM DESIGN INTERVIEW

Design a Distributed Message Queue

Kafka-style partitioned log with brief comparison to RabbitMQ, SQS, and Pub/Sub

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Topic & Partition Model
- 06** · Storage: The Append-Only Log
- 07** · Replication & ISR
- 08** · Producer Path
- 09** · Consumer Groups & Rebalance
- 10** · Delivery Semantics

- [11](#) · [Comparison: Kafka vs RabbitMQ vs SQS vs Pub/Sub](#)
- [12](#) · [Operations & Observability](#)
- [13](#) · [Failure Modes & Recovery](#)
- [14](#) · [Design Trade-offs](#)
- [15](#) · [Interview Playbook](#)

01

Problem Statement

Requirements and scope

Design a distributed message queue like **Apache Kafka**. The system must accept events from many producers, persist them durably, and deliver them to many consumers in order, at high throughput, with tunable delivery semantics — all while surviving broker failures and network partitions.

Clarifying Questions

Question	Answer
Peak throughput?	10M messages/sec across cluster (1M sustained)
Avg message size?	1 KB (range: 100 B to 1 MB)
Retention?	7 days default; configurable up to 30 days
Ordering?	Per-partition (per-key) ordering, not global
Delivery semantics?	Configurable: at-most, at-least, exactly-once
Multi-tenant?	Yes; quotas per producer / consumer group
Geo-replicated?	Single region first; cross-region later (MirrorMaker)
Use-cases?	Event streaming, log aggregation, CDC, async services

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Publish	Producer sends record to topic; gets ack with offset
Subscribe	Consumer joins group; auto-rebalances partitions
Retention	Time-based (7d) or size-based (e.g. 1 TB/partition)
Replay	Consumers can seek to any offset within retention
Ordering	FIFO within a partition (key-based hashing)
Compaction	Optional log compaction by key (latest value wins)
ACL / quotas	Per-topic ACLs; per-client byte/sec quotas

Non-Functional Requirements

Requirement	Target
Throughput	10M msg/sec peak; 1M sustained (cluster-wide)
Latency	p50 <10 ms publish; p99 <100 ms end-to-end
Durability	No data loss with acks=all + min.insync=2
Availability	99.99% (multi-AZ); cluster survives 1 broker loss
Retention	7 days default → ~600 TB/cluster
Scalability	Add brokers / partitions linearly

03

Capacity Estimation

Math for scale

Throughput

- Peak ingest: **10M msg/sec** × 1 KB = **10 GB/sec** raw
- Sustained ingest: **1M msg/sec** × 1 KB = **1 GB/sec**
- Read amplification: typically 3 consumer groups × producers ≈ **3 GB/sec out**
- Replication amplification: factor 3 (1 leader + 2 followers) ⇒ inter-broker traffic ≈ 2 GB/sec

Storage (7-day retention)

- If **10M/sec** retained **7 days**: 10 GB/s × 86,400 × 7 ≈ **6 PB** raw, ×3 RF = **~18 PB** on disk
- Annualized at peak: 10 GB/s × 365 × 86,400 ≈ **~315 PB/yr** raw (~80 PB/yr usable after 4× index/overhead at 7-day rolling)
- **Realistic (1M sustained)**: 1 GB/s × 86,400 × 7 ≈ **600 TB** raw, ×3 RF = **~1.8 PB**
- Per broker (30 brokers): ~60 TB/broker — fits 12× 8 TB HDD JBOD comfortably

Partitioning

- Per-partition write throughput ceiling: **~50 MB/s** (sequential disk + replication)
- Sustained 1 GB/s ÷ 50 MB/s ≈ **20 partitions minimum**; pick **200** for headroom + parallelism
- Peak 10 GB/s ÷ 50 MB/s = **~200 partitions** minimum at peak; recommend **500–1000**
- **Brokers**: 30–50 brokers; each owns ~20–30 partition leaders

KEY

Key Numbers

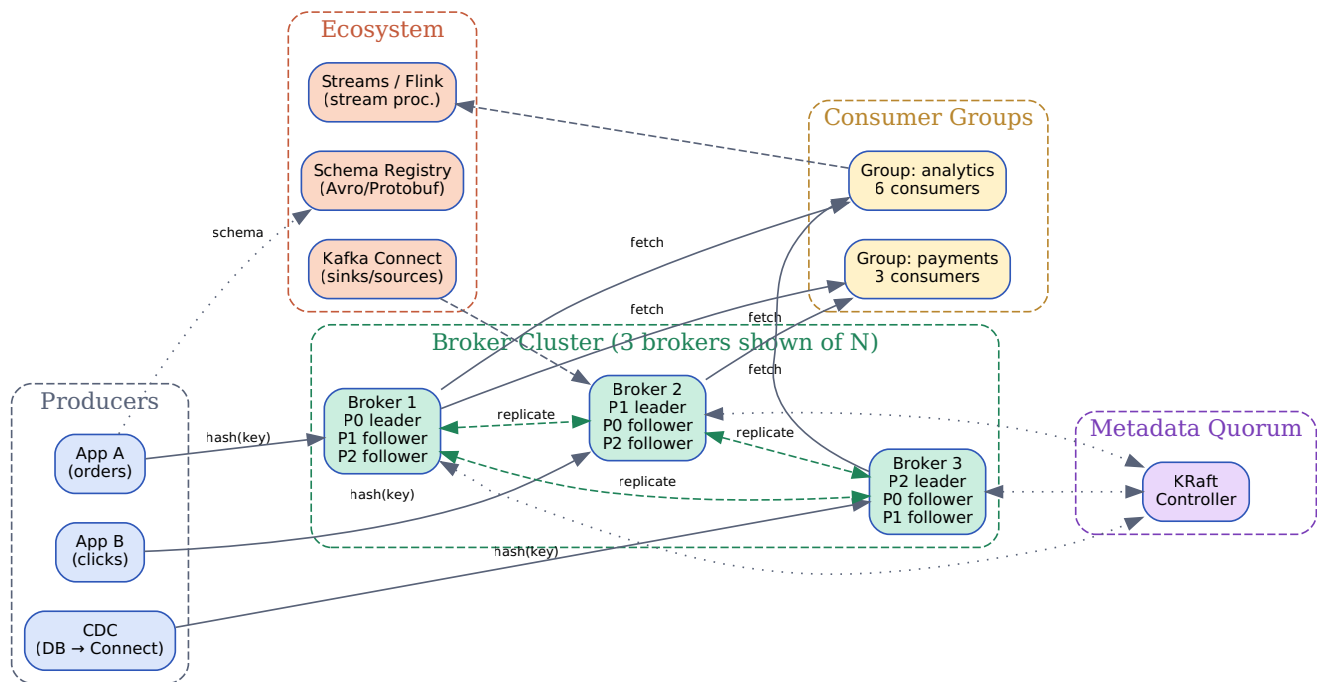
Peak: **10M msg/sec** = **10 GB/sec** · Sustained: **1M msg/sec** = **1 GB/sec** · 7-day storage: **~600 TB**
sustained, **~6 PB** peak · RF=3 · **~500 partitions**, **~30 brokers**.

04

High-Level Architecture

System overview

Producers publish records to **topics**, which are split into **partitions** distributed across a **broker cluster**. Each partition has one leader and N followers replicated via the **ISR** (In-Sync Replica) protocol. Consumers join **consumer groups** and each partition is assigned to exactly one consumer in the group. A coordinator (KRaft / Raft quorum, replacing legacy ZooKeeper) manages metadata, leader election, and group membership.



Producers hash by key into partitioned topics on a replicated broker cluster; consumer groups read independently. KRaft quorum holds metadata.

Architecture Highlights

- **Producers:** batch & compress records; route by `hash(key) % num_partitions`
- **Brokers:** stateless protocol over a stateful append-only log on disk
- **Partitions:** unit of parallelism, ordering, and replication
- **ISR replication:** leader + followers; only in-sync replicas can be elected
- **KRaft controller:** Raft quorum holds metadata (replaces ZooKeeper since Kafka 3.3)
- **Consumer groups:** partition-to-consumer assignment; offset commits per partition
- **Ecosystem:** Connect (ETL), Streams/Flink (processing), Schema Registry (contracts)

05

Topic & Partition Model

How records are addressed

The Hierarchy

- **Topic** = named, ordered, append-only stream (e.g. `orders.v1`)
- **Partition** = a sub-log of a topic; records strictly ordered within it
- **Offset** = monotonically increasing 64-bit ID of a record within a partition
- **Record** = (key, value, headers, timestamp). Key drives partition routing.

Partition Assignment by Producer

- **If key present:** `partition = murmur2(key) % num_partitions` — same key always routes to same partition (preserves ordering)
- **If key absent:** sticky partitioner — batches records to one partition per linger window for throughput
- **Custom partitioner:** implement `Partitioner` interface for tenant isolation, hot-key spreading, etc.
- **Hot key warning:** celebrity-key skew breaks partition parallelism; use composite key or salted prefix

Choosing Partition Count

Driver	Rule of Thumb	Why
Throughput	$\geq \text{target_MBps} / 50 \text{ MB/s/partition}$	Per-partition cap from disk + replication
Consumer parallelism	$\geq \text{max consumers in any group}$	Each partition = 1 consumer; extras idle
Ordering	$\geq N$ where $N = \text{ordering scope count}$	All records of one key share a partition
Cost	$\leq 4,000$ per broker, $\leq 200,000$ cluster	Each partition costs file handles + memory
Future-proof	Pick 2–3× current need	Reducing partitions later requires recreate

WARNING

Repartitioning Is Painful

Adding partitions changes the hash output for existing keys, breaking ordering for those keys going forward. Plan partition count generously up front, or design downstream consumers to tolerate key-route changes during migration windows.


```
00000000000001048576.index  
00000000000001048576.timeindex
```

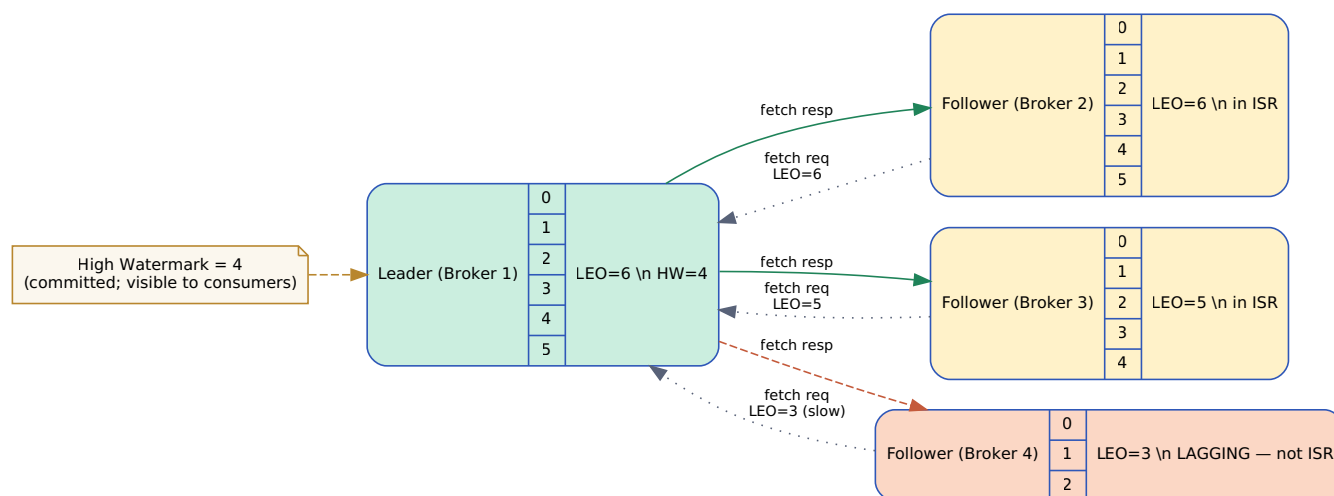
07

Replication & ISR

Durability through followers

Leader / Follower / ISR

- Each partition has **one leader + RF – 1 followers** on different brokers
- Producers write only to the leader; followers *fetch* from the leader (pull, not push)
- **In-Sync Replica (ISR)**: a follower that has caught up to within `replica.lag.time.max.ms` (default 30 s)
- Only an ISR member is eligible for leader election (under default safe config)
- **High Watermark (HW)**: highest offset replicated to all ISRs — only HW-and-below records are visible to consumers
- **Log End Offset (LEO)**: next free offset on each replica



Leader holds full log; followers fetch and ack. High Watermark = $\min(\text{LEO across ISRs})$ — only committed records visible to consumers.

acks: Producer Durability Knob

acks	Behavior	Latency	On Loss
0	Fire and forget; no broker ack	Lowest	Drops on broker crash, network drop
1	Leader writes locally then acks	Low	Loses if leader crashes before replication
all (-1)	Wait for all ISRs to replicate	Highest	No loss as long as $\geq \text{min.insync.replicas}$ alive

min.insync.replicas & Unclean Election

- **min.insync.replicas=2** with RF=3 + acks=all: tolerate 1 broker loss, never lose data
- If ISR shrinks below this, producer gets `NotEnoughReplicas` — fail loud, don't lose
- **unclean.leader.election.enable=false** (default): if all ISRs die, partition stays unavailable until one returns
- **unclean=true**: a non-ISR replica may be elected leader → *data loss possible*, but availability restored sooner

WARNING

The Classic Trade-off

RF=3, min.insync=2, acks=all, unclean=false ⇒ **maximum durability**. But two simultaneous follower failures stall writes until one recovers. Lowering min.insync to 1 or enabling unclean election trades correctness for availability — explicit business decision.

08

Producer Path

Batching, idempotence, transactions

Producer Lifecycle

1. App calls `send(record)` — non-blocking, returns a `Future<RecordMetadata>`
2. Record assigned to a partition (key hash or sticky)
3. Appended to per-partition **RecordAccumulator** batch buffer in memory
4. Sender thread waits up to `linger.ms` or until batch fills `batch.size`
5. Batch compressed (lz4 / zstd / gzip / snappy) and sent in one `ProduceRequest` to the partition leader
6. Leader appends, awaits ISR replication if `acks=all`, then responds with `RecordMetadata{offset, partition}`
7. Future completes; callback fires with offset or error

Producer Configuration (Java)

```
Properties props = new Properties();
props.put("bootstrap.servers", "b1:9092,b2:9092,b3:9092");
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "io.confluent.kafka.serializers.KafkaAvroSerializer");

// Durability
props.put("acks", "all");
props.put("enable.idempotence", "true"); // dedup retries
props.put("max.in.flight.requests.per.connection", "5"); // safe with idempotence
props.put("retries", "2147483647");
props.put("delivery.timeout.ms", "120000");

// Throughput
props.put("linger.ms", "10"); // wait up to 10 ms to batch
props.put("batch.size", "131072"); // 128 KB per partition batch
props.put("compression.type", "lz4"); // ~3-4x compression on JSON
props.put("buffer.memory", "67108864"); // 64 MB total buffer

// Exactly-once across topics (transactions)
props.put("transactional.id", "orders-svc-1");

Producer<String, Order> p = new KafkaProducer<>(props);
p.initTransactions();
p.beginTransaction();
p.send(new ProducerRecord<>("orders.v1", order.userId(), order));
```

```
p.send(new ProducerRecord<>("audit.v1", order.userId(), audit));  
p.commitTransaction(); // or abortTransaction() on failure
```

Idempotent Producer

- Each producer assigned a **PID** (Producer ID) on init
- Each (PID, partition) carries a monotonic **sequence number**
- Broker dedups on (PID, partition, seq); retried sends after a successful but unacked write don't duplicate
- **Cost:** ~1% throughput; gives *per-session* exactly-once into one partition

Backpressure

- **buffer.memory** bound: when full, `send()` blocks up to `max.block.ms`, then throws
- **Broker quotas:** per-client byte/sec; broker throttles by delaying responses (no errors thrown)
- **Linger trade-off:** `linger.ms=0` $\Rightarrow$ low latency, low throughput; `linger.ms=20` $\Rightarrow$ ~10 $\times$ throughput, +20 ms p50

09

Consumer Groups & Rebalance

Cooperative partition ownership

The Group Protocol

- Consumers join a **group.id**; broker-side **Group Coordinator** assigns partitions
- **Invariant**: each partition is consumed by exactly one consumer in the group at a time
- If `num_consumers > num_partitions`: extra consumers stay idle (still good for HA)
- Multiple groups consume the same topic independently — each has its own offset cursor
- **Assignment strategies**: Range, RoundRobin, Sticky, **CooperativeSticky** (incremental rebalance)

Offset Commits

- Committed offsets stored in internal topic `__consumer_offsets` (50 partitions, RF=3)
- **Auto-commit** (every 5 s): simple, but at-least-once with potential reprocessing on crash
- **Manual commit**: `commitSync()` (blocks, retries) or `commitAsync()` (fire-and-forget)
- **Best practice**: process record → commit; if process is non-idempotent, design downstream to dedup

Rebalance Algorithm (Pseudocode)

```
# Cooperative-sticky rebalance (no full stop-the-world)
# Triggered by: new consumer joining, heartbeat timeout, topic-partition added

function on_join_group(group_id, member_id):
    coordinator = locate_coordinator(group_id)
    coordinator.send(JoinGroupRequest{member_id, subscribed_topics})
    # Coordinator picks one member as 'leader' (first to arrive)
    response = coordinator.recv()
    if response.is_leader:
        new_assignment = sticky_assign(
            members      = response.members,
            partitions    = topic_metadata.all_partitions(),
            previous      = response.current_assignment,
        )
        coordinator.send(SyncGroupRequest{assignments=new_assignment})
    else:
        coordinator.send(SyncGroupRequest{})
        my_assignment = coordinator.recv().assignment
        revoked = old_assignment - my_assignment
        added   = my_assignment - old_assignment
        on_partitions_revoked(revoked)    # commit offsets, flush state
        on_partitions_assigned(added)    # seek to last committed offset
```

```
# Heartbeat thread keeps membership alive every 3 s
function heartbeat_loop():
    while running:
        coordinator.send(HeartbeatRequest{member_id, generation_id})
        if response.error == REBALANCE_IN_PROGRESS:
            trigger_rejoin()
        sleep(heartbeat_interval_ms)

# Fetch / commit happens between heartbeats
function poll_loop():
    while running:
        records = coordinator.fetch(my_assignment, last_committed_offsets)
        for r in records:
            process(r)                # idempotent ideally
        coordinator.commit(highest_offset_per_partition)
```

TIP

Avoiding Rebalance Storms

Each rebalance pauses processing on revoked partitions. Tune `session.timeout.ms` and `max.poll.interval.ms` above expected GC pauses. Use **CooperativeStickyAssignor** (default since Kafka 3.0) for incremental rebalances — only the moving partitions pause, not the whole group.

10

Delivery Semantics

At-most, at-least, exactly-once

Semantic	Producer	Consumer	Use Case
At-most-once	acks=0/1, no retries	commit before process	Metrics; loss tolerable
At-least-once	acks=all, retries on	process before commit	Default; downstream dedup
Exactly-once (in-Kafka)	idempotent + transactional.id	isolation.level=read_committed	Stream-to-stream pipelines
Effectively-once (end-to-end)	at-least-once + idempotent sink	dedupe by key/seq downstream	Writes to DB, external API

How Exactly-Once Works (Two-Phase Commit)

```
# Producer transaction across multiple partitions / topics
# (atomic from the consumer's read_committed view)

function consume_process_produce_loop():
    producer.init_transactions()                # registers txn.id with TxnCoordinator
    while running:
        records = consumer.poll(timeout=100ms)
        if records.empty(): continue

        producer.begin_transaction()
        try:
            for r in records:
                out = transform(r)
                producer.send('out_topic', out)    # buffered in txn

            # Critical: commit consumer offsets WITHIN the producer txn
            offsets = consumer.position_per_partition()
            producer.send_offsets_to_transaction(
                offsets, consumer.group_metadata()
            )
            producer.commit_transaction()          # 2PC: prepare + commit markers
        except:
            producer.abort_transaction()
            consumer.seek(last_committed_offsets)  # rewind

# Broker side:
```

```
# 1. Producer sends records with txn.id – broker buffers them in the log,  
#    marked uncommitted (not visible to read_committed consumers).  
# 2. commit_transaction() -> TxnCoordinator writes commit marker to each  
#    involved partition (including __consumer_offsets).  
# 3. Once marker written, records become visible. Atomic across partitions.
```

Trade-offs

- **At-most-once:** simplest; *only* for replaceable signals (heartbeats, sampled metrics)
- **At-least-once:** cheap and reliable — most pipelines run here with idempotent consumers
- **Exactly-once:** ~10–30% throughput hit + complexity; only worth it for in-Kafka stream processing chains
- **End-to-end EOS to a DB:** Kafka's transaction does NOT cover the external sink; need outbox / idempotent upsert

11

Comparison: Kafka vs RabbitMQ vs SQS vs Pub/Sub

Picking the right tool

Aspect	Kafka	RabbitMQ	AWS SQS	Google Pub/Sub
Model	Partitioned log (pull)	AMQP broker (push)	Distributed queue	Topic + sub (pull/push)
Ordering	Per-partition FIFO	Per-queue FIFO (single consumer)	FIFO queues only (limited TPS)	Per-key with ordering keys
Throughput	Millions/sec/cluster	10s of K/sec/node	~3K/sec/queue (FIFO 300/sec)	Millions/sec (managed)
Retention	Days–years; replay	Until consumed (default)	1 min – 14 days	Up to 31 days; replay via snapshots
Delivery	At-least, exactly-once	At-least; at-most	At-least (visibility timeout)	At-least; exactly-once (regional)
Routing	Hash partition by key	Exchanges + routing keys (rich)	Flat queue; one consumer pool	Topic → many subs (fan-out)
Strength	High-throughput log, replay	Complex routing, low latency	Zero-ops AWS native	Zero-ops GCP native, global
Weakness	Ops complexity; partition planning	Storage on broker; harder to scale	No replay; small messages only (256 KB)	Vendor lock-in; cost at scale

When to Pick What

- **Kafka:** event sourcing, log aggregation, stream processing, CDC, anywhere replay matters
- **RabbitMQ:** task queues with priority/TTL/DLX, complex routing topologies, request/reply
- **SQS:** simple decoupling on AWS, work queues with visibility timeout, no replay needed
- **Pub/Sub:** fan-out events across services on GCP, push to HTTP/Cloud Run, global delivery

INSIGHT

Push vs Pull

RabbitMQ *pushes* to consumers (broker-driven flow). Kafka and SQS *pull* (consumer-driven). Pull naturally handles backpressure — slow consumers just lag — and lets brokers stay simple, append-only servers. Push needs broker-side flow control (prefetch, credit) to avoid overwhelming consumers.

12

Operations & Observability

Running the cluster

Topic Lifecycle (CLI)

```
# Create a topic with explicit replication and partition count
kafka-topics.sh --bootstrap-server b1:9092 \
  --create --topic orders.v1 \
  --partitions 200 --replication-factor 3 \
  --config min.insync.replicas=2 \
  --config retention.ms=604800000 \
  --config compression.type=producer \
  --config segment.bytes=1073741824

# Inspect a consumer group's lag
kafka-consumer-groups.sh --bootstrap-server b1:9092 \
  --group payments --describe
# GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
# payments orders.v1 0           1432901         1432955         54   pay-pod-7
# payments orders.v1 1           1428770         1432892         4122 pay-pod-3 <-- hot!

# Reassign partitions when adding brokers (rebalance)
kafka-reassign-partitions.sh --bootstrap-server b1:9092 \
  --reassignment-json-file plan.json --execute
```

Key Metrics to Alert On

Metric	Healthy	Page When
UnderReplicatedPartitions	0	> 0 sustained > 5 min
OfflinePartitionsCount	0	> 0 — partition unavailable
ActiveControllerCount	1 (cluster-wide)	≠ 1 — split brain risk
RequestHandlerAvgIdlePercent	> 30%	< 20% — broker saturated
NetworkProcessorAvgIdlePercent	> 30%	< 20% — network saturated
Consumer lag (per group)	Bounded, decreasing	Growing > 10 min
ISR shrinks/sec	~0	Spikes — flaky followers

Metric	Healthy	Page When
Disk usage	< 70%	> 80% — retention or scale up

Capacity Planning Heuristics

- **Brokers:** $N = \text{peak_MBps_in} \times \text{RF} / \text{per-broker-cap}$. With 200 MB/s/broker cap and $10 \text{ GB/s} \times 3 = 30 \text{ GB/s} \Rightarrow 150$ brokers at peak; 30 brokers at sustained.
- **Partitions per broker:** $\leq 4,000$; at ~ 500 partitions $\times \text{RF } 3 = 1500$ replicas / 30 brokers = 50/broker — very comfortable
- **Disk:** $\text{retention_GB} / \text{num_brokers} + 30\% \text{ headroom}$; plan re-replication time
- **Network:** producers $\rightarrow$ leader = $1\times$, leader $\rightarrow$ followers = $(\text{RF}-1)\times$ — biggest pipe is replication

13

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Broker process crash	Followers it led re-elect; brief unavailability per partition	ZK/KRaft session timeout	Auto failover; replay from disk on restart
Disk failure / corruption	Replica offline; ISR shrinks	Read errors; log corruption marker	Reformat + re-replicate from leader (hours at TB scale)
Network partition	Some brokers think others dead; split risk	Heartbeat timeouts; ISR churn	KRaft quorum prevents split brain; min.insync gates writes
Slow follower	ISR shrinks; throughput drops if min.insync=RF	replica lag metric	Dial replica.lag.time.max; investigate disk/GC
Slow consumer	Group lag grows; possible rebalance	Consumer lag metric	Scale consumers; tune max.poll.records / interval
Hot partition (key skew)	One broker saturated; head-of-line blocking	Per-partition byte rate	Re-key with salt; spread across more partitions
Producer overflow	send() blocks/throws BufferExhausted	buffer-available-bytes	Increase buffer.memory; lower throughput; quotas upstream
Schema break	Consumers can't deserialize	Deserialization errors	Schema Registry compatibility checks (BACKWARD)
KRaft quorum loss	No metadata changes; existing partitions still read/write	Controller election fails	Restore quorum majority; periodic backups of metadata log

Disaster Recovery

- **Multi-AZ:** spread replicas via broker . rack; rack-aware assignment guarantees one replica per AZ
- **Cross-region:** MirrorMaker 2 (async); typically active/passive — RPO seconds, RTO minutes
- **Tiered storage** (Kafka 3.6+): offload old segments to S3 — cheaper retention, faster broker recovery
- **Backup of __consumer_offsets:** include in MM2 mirroring so failover preserves consumer cursors

WARNING

Re-replication Is Slow

Losing a broker that held 60 TB means the cluster must copy ~60 TB across the network to bring replicas back to RF=3. At 1 Gbps that's ~6 days; at 10 Gbps ~14 hours. During that window you have reduced redundancy. **Tiered storage** dramatically shrinks recovery because the broker only re-replicates the local hot tail.

14

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Durability vs latency	acks=all + min.insync=2	+5–20 ms p99 vs acks=1, but no loss on single broker failure.
Replication factor	RF=3	3× storage cost; tolerates 1 broker loss with min.insync=2. RF=2 risky; RF=5 expensive.
Partition count	500 (10× peak need)	More parallelism but more file handles + rebalance time. Repartitioning is hard, so over-provision.
Retention	7 days time-based	Enables replay/backfill; ~600 TB cost. Longer = more storage; shorter = no recovery window.
Storage media	HDD JBOD	5–10× cheaper than SSD; sequential I/O hides HDD latency. SSD only if random reads dominate.
Compression	lz4 producer-side	~3–4× shrink with small CPU cost; gzip = better ratio, more CPU; zstd = best modern balance.
Delivery semantics	At-least-once + idempotent consumers	Simpler than EOS; downstream must dedup. EOS only for in-Kafka stream chains.
Push vs pull	Pull	Consumers control flow; brokers stay simple/stateless protocol; modest extra fetch RTTs.
Broker controller	KRaft (Raft quorum)	One fewer system to run vs ZooKeeper; younger code path.
Message format	Avro + Schema Registry	Compact, evolvable; vs JSON's human-readability or Protobuf's static typing.

15

Interview Playbook

How to present this

A 45-minute interview on this topic almost always anchors on Kafka's partitioned-log model. Start with throughput math, derive partitions and brokers from it, then drill into ISR replication, delivery semantics, and consumer groups. Have the comparison table ready for the inevitable “why not RabbitMQ?” follow-up.

45-Minute Interview Outline

1. **Intro (2 min):** clarify peak vs sustained, retention, ordering scope, semantics needed
2. **Capacity (5 min):** 10M/s peak → 10 GB/s in, ×3 RF replication, 600 TB at 7-day sustained
3. **High-level (3 min):** producers → topics/partitions on broker cluster → consumer groups; KRaft for metadata
4. **Storage (5 min):** append-only segmented log, sequential I/O, page cache, sendfile zero-copy
5. **Replication (8 min):** ISR, leader/follower, HW vs LEO, acks, min.insync, unclean election
6. **Producer (4 min):** batching/linger, idempotence (PID + seq), transactions
7. **Consumer groups (6 min):** partition-per-consumer invariant, rebalance, offset commits
8. **Semantics (4 min):** at-most/at-least/exactly-once; transactional 2PC across topics + offsets
9. **Compare (4 min):** Kafka vs RabbitMQ vs SQS vs Pub/Sub — pick the right tool
10. **Failures + trade-offs (4 min):** hot partition, slow consumer, broker loss, network partition

Common Follow-ups & Answers

- **Q: Why is Kafka so fast on commodity disks?** A: Append-only sequential I/O (~600 MB/s on HDD), kernel page cache for tail reads, zero-copy `sendfile` avoids user-space copies.
- **Q: How is ordering preserved?** A: Per-partition FIFO. Producers route by `hash(key) % partitions`, so same key = same partition. There is no global order.
- **Q: What if a leader crashes mid-write?** A: Controller elects a new leader from the ISR. With `acks=all + min.insync=2`, the writes are safe on at least 2 replicas before ack. Followers truncate uncommitted suffix on becoming leaders.
- **Q: Exactly-once explained?** A: Idempotent producer (PID + sequence dedup) + `transactional.id` wrapping `send_offsets_to_transaction` so consumer offsets and produced records commit atomically (2PC via `TxnCoordinator`).
- **Q: When NOT to use Kafka?** A: When you need rich routing (use RabbitMQ), when ops budget is zero (use SQS/Pub-Sub), when retention < minutes and message size > MB (use object store + queue).
- **Q: How do you handle a hot key?** A: Composite key (key + bucket suffix) to spread across N partitions, accepting per-bucket ordering instead of per-key. Or move that key to a dedicated topic.

- **Q: Why pull instead of push?** A: Consumers absorb backpressure naturally (just lag); brokers stay simple servers; batch sizes can match consumer capability.

Key Talking Points

- “10 GB/s peak ÷ 50 MB/s/partition ⇒ ~200 partitions floor; pick 500 for headroom”
- “RF=3 + min.insync=2 + acks=all = no data loss with 1 broker down”
- “Sequential disk + page cache + sendfile = why HDDs are competitive”
- “Per-partition order, not global — by design”
- “At-least-once + idempotent sinks beats exactly-once in most pipelines”
- “KRaft replaced ZooKeeper — one fewer system to run”

KEY

Memorize These Numbers

Peak **10M msg/sec** · Sustained **1M msg/sec** · Avg msg **1 KB** · **50 MB/s/partition** ceiling · **RF=3**,
min.insync=2 · **500 partitions, 30 brokers** · **~600 TB** at 7-day sustained · HDD sequential **~600 MB/s**
· linger.ms=10, batch.size=128 KB.